Video-to-3D Animation Tool — Development Plan & Software Specification

Nov 8, 2026 · @Steve

Build a personal pipeline that turns boxing, MMA and kung fu footage into game-ready fighting animation, in about 27 part-time weeks. It is a Python command-line tool on your NVIDIA GPU plus a Blender add-on for cleanup. Exports go to Unity, Unreal or Godot.

The core model is NVIDIA's GEM-X, released 2026-10-30, which produces world-grounded motion for each fighter. Like every model on the export path, its licence allows commercial use of the output, so the animations can ship in a game you sell. Research models built on the non-commercial SMPL body model are not used.

The first usable version, on 2027-03-28, solves paired attacker and defender exchanges into your engine. The full tool, with the Blender cleanup add-on, follows on 2027-06-20. Both dates assume a steady 12 hours a week.

The main risks:

- **Legal.** A sold game could contain motion copied from film choreography. Mitigations are recording where each take came from, preferring sports footage, and reworking film moves.
- **Model quality.** GEM-X is brand new and may struggle with overlapping, blurred broadcast fighters.
- **Windows setup.** The models expect Linux, so their environments run in WSL2.

Phase 0 (2026-12-01 to 12-20) tests all three on real fight shots before anything is built around them.

Goals, scope and non-goals

The tool is a personal pipeline. It extracts fighting animation (boxing, MMA striking, kung fu) from existing footage, such as broadcast fights and martial-arts films, for use in your own game projects. The output is what matters, not the tool as a product, so it is not packaged, sold or supported for other people.

This footage is harder than self-filmed video. Shots last a few seconds, cameras pan and zoom, two fighters (often plus a referee) overlap constantly, and motion is fast and blurred. Expect reference-grade motion that needs cleanup, not finished animation straight from the solver.

Goals

ID	Goal	Measure of success
G-1	Turn a clean striking exchange from a broadcast or film into a usable game animation	A 3–6 s exchange (jab-cross, kick, combination, dodge) becomes an in-engine animation on your character after ⩽ 30 min of cleanup
G-2	Capture both fighters of an exchange	Attacker and defender are solved in one shared world, with consistent spacing and facing, so paired attack/react animations line up
G-3	Mine long videos quickly	From a 20-minute fight or a 90-minute film, you can mark and queue 20 candidate shots in ⩽ 15 min, then process them unattended
G-4	Fix motion at the level of intent	Foot skate, jitter and bad frames are fixed with contact painting, smoothing and pinned poses, not curve by curve
G-5	Drop-in export to your engine	FBX or glTF imports into Unity, Unreal or Godot on a Mixamo/Humanoid-style rig with no manual renaming; root motion works
G-6	Keep it buildable by one person	A working footage-to-engine path by the end of P2; each phase is usable on its own

Primary use cases, ranked

1. **Stand-up striking from boxing and MMA broadcasts.** Punches, kicks, footwork, slips and guards from wide or mid shots with both fighters fully visible.
2. **Kung fu film choreography.** Longer, staged exchanges filmed for readability, but often undercranked (sped up), wire-assisted or cut on impact.
3. **Hit reactions and paired moves.** The defender's reaction, solved in the same world as the attacker for synchronised game animations.
4. **Slow-motion replays.** Broadcast replays give high temporal detail of a single strike once retimed to real speed.

5. **Later: clinch and grappling.** Takedowns and ground work. Current models fail badly on entangled bodies, so this is a stretch goal with low expectations.

Scope

Area	First usable version (end of P2)	Full version (end of P4)	Backlog
Input	Local video files; a shot list marked from a long video; cuts detected	Letterbox and overlay handling; retime for slow-motion and undercranked film	Linking one fighter across shots
People	Two fighters per shot, referee and crowd ignored	Inter-person penetration fix for close exchanges	Clinch and grappling
Camera	Pans and zoom (focal length per frame)	Better stabilisation on handheld film shots	—
Body	World-grounded full body, root trajectory, contacts	Strike-aware smoothing tuned on the test set	Physics plausibility pass
Hands and face	Fist pose preset for gloved or closed hands	Fingers for open-hand kung fu forms	Face (rarely visible enough)
Cleanup	Smoothing and contact settings via CLI	Blender add-on: overlay, contacts, pins, re-run range	Loop and blend helper
Export	FBX and BVH on a Mixamo-style rig, attacker and defender as separate takes	glTF; Unity, Unreal and Godot presets; retargeting done in the engine	Live preview in the engine
Platform	Windows 11, NVIDIA GPU	Same	Linux

Non-goals

- **No video downloading.** The tool takes local video files, and how you obtain footage is outside it (see Risks).
- **No desktop app of its own.** Blender is the review and cleanup UI.
- **No server, accounts, installer or telemetry.**
- **No in-house model training or mocap-lab capture.**

- **No reliable grappling or ground fighting in v1.**
- **No real-time capture.**

Requirements

There are 21 functional requirements: 13 Must, 5 Should and 3 Could. Twelve non-functional targets go with them. The Musts that fight footage adds are a shot list, two-fighter solving, zoom-aware camera estimation and retiming. Processing speed matters less than cleanup speed, because shots are queued and solved unattended.

Functional requirements

ID	Requirement	Priority	Phase
FR-1	Detect shot cuts in a long video and build a shot list; mark in/out ranges by hand; queue chosen shots	M	P1
FR-2	`afv run` processes a queued shot end to end into one take per selected fighter	M	P1
FR-3	Detect and track everyone; pick the fighters by track ID; ignore referee and crowd; hold IDs through clinches and crossings (with manual swap fix)	M	P1
FR-4	Estimate camera motion and a per-frame focal length, so broadcast pans and zooms are separated from fighter motion	M	P1
FR-5	Solve world-grounded 3D body motion for each fighter in one shared world frame	M	P1
FR-6	Detect foot contacts and remove foot skate and ground penetration on a single floor plane (ring canvas, cage mat, film set)	M	P1
FR-7	Smooth jitter with a strength setting (global and per region), and turn smoothing down automatically around detected strikes so impacts stay sharp	M	P1

FR-8	Cache every stage; re-run from any stage on all frames or a range	M	P1
FR-9	Retime: a speed factor per shot for slow-motion replays and undercranked film, output at the game's frame rate	M	P2
FR-10	Convert SOMA motion to one canonical humanoid export rig (Mixamo-compatible names, T-pose rest, optional root bone); your characters are retargeted in the engine	M	P2
FR-11	Export FBX and glTF/GLB with in-place, world or extracted root motion; attacker and defender as separate takes with a shared origin; BVH for debugging	M	P2
FR-12	Fist hand pose preset for gloved or closed hands	M	P2
FR-13	Blender add-on: import takes, video as camera background, confidence strip, fix ID swaps, paint contacts, pin poses, re-run a range	M	P3
FR-14	Mask burned-in overlays (scoreboards, logos, subtitles) and crop letterbox before detection	S	P2
FR-15	Inter-person penetration fix so gloves and limbs don't pass through the opponent	S	P4
FR-16	High mode: fuse SAM 3D Body for small or blurred fighters (research, no published numbers)	C	Backlog
FR-17	Finger motion for open-hand forms (toggle)	S	P4
FR-18	Batch mode: process the whole queue overnight with saved settings	S	P1
FR-19	Shot browser in Blender: scrub a long video, see detected cuts, add shots to the queue	S	P3
FR-20	Loop and blend helper for idles and footwork cycles	C	Backlog
FR-21	Clinch and grappling support	C	Backlog

Non-functional requirements

These assume an RTX 3060 12 GB or better. The GPU in your PC still needs confirming (see Risks).

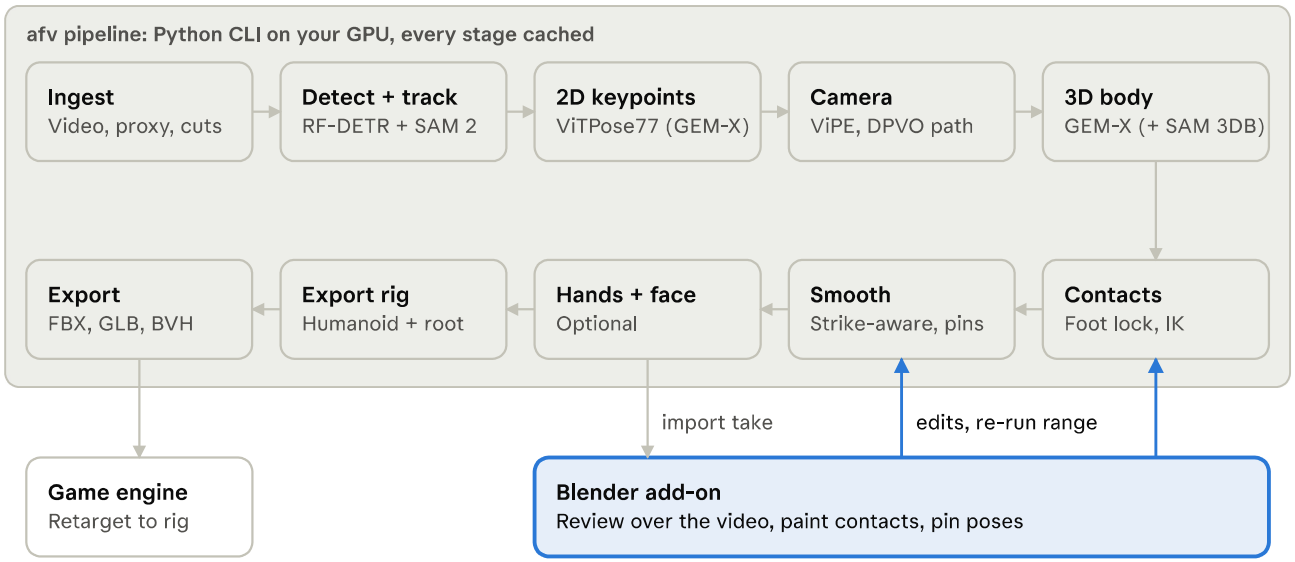
ID	Category	Target
NFR-1	Throughput	A 5 s 1080p shot with two fighters: ⩽ 3 min with `--static-camera`, ⩽ 6 min with camera estimation, on an RTX 4070-class GPU; a queue of 20 shots finishes overnight. Measured on your GPU in P0, then fixed
NFR-2	Partial re-run	A re-run from smoothing or contacts on a 2 s range takes ⩽ 30 s; a smoothing change previews in Blender in ⩽ 5 s
NFR-3	Foot skate	Mean foot speed during detected contact ⩽ 3 cm/s after cleanup
NFR-4	Jitter	No visible single-frame pops at default smoothing, while strike peaks keep ⩾ 80% of their unsmoothed speed
NFR-5	Pose fidelity	Solved joints reprojected onto the video ⩽ 15 px mean at 1080p on visible joints; motion judged "right" by eye on the test set
NFR-6	Shot length	Shots from 1 s to 60 s; source videos up to 2 h (cut detection runs on the 720p proxy)
NFR-7	Input	MP4, MKV and WebM with H.264, VP9 or AV1; 480p–4K; 23.976–60 fps; fighters ⩾ 150 px tall usable, ⩾ 300 px recommended
NFR-8	Platform	Windows 11 with WSL2 Ubuntu for the model environments; NVIDIA GPU with ⩾ 12 GB VRAM; Blender 5.2 LTS (pinned build)
NFR-9	Privacy	Runs fully offline once models are installed; nothing is uploaded
NFR-10	Reproducibility	Same shot, settings and model versions give the same result, within float tolerance

NFR-11	Export fidelity	Exported clips play in Blender and your engine with correct scale, axes, frame count and root motion; checked by a scripted round-trip test
NFR-12	Licence hygiene	Every checkpoint on the export path, including transitive downloads, records its code licence, weight licence and training-data terms. Export requires code and weight licences that allow commercial use; training-data terms are reviewed before any game is sold

Architecture and processing pipeline

The system has two parts. A Python package, `afv`, runs the ML pipeline from the command line on your GPU. A Blender add-on is the review and cleanup interface. Every stage writes a cached result to the project folder, so a fix in Blender re-runs only the stages after it and only on the affected frames.

A fix in Blender re-runs only the stages after it



The top row turns video into world-grounded body motion. The bottom row cleans it and fits it to your rig. Contact and pin edits made in Blender re-enter the pipeline at the contacts or smooth stage, so those fixes never re-run the expensive models above them.

Components

Component	What it does	Built on
`afv` CLI and library	Runs stages, manages the cache, prints progress (human-readable or JSON lines)	Python 3.11, Typer
Model environments	One pinned environment per heavy model (`gemx` , `vipe` , `sam3db`), each called as a subprocess through the stage interface	WSL2 Ubuntu + CUDA 12.8 by default; Docker images where the repo provides them
Stage plugins	One adapter per stage; models are swapped by changing a preset	Each model's repo, pinned by commit and weight hash
Project folder	Holds the manifest, stage results, edits and exports per shot	JSON + NPZ (see Data model)
Media layer	Decodes video, keeps true timestamps, builds a 720p proxy, detects cuts	FFmpeg 7 via PyAV
`afv serve` worker	A long-lived process the Blender add-on talks to, so each fix avoids paying Python and CUDA start-up again	JSON-RPC over localhost
Export	Builds the canonical humanoid export rig and writes FBX/glTF/BVH	Headless Blender 5.2 LTS (pinned build); own BVH writer
Blender add-on	Shot browser, take import, video overlay, edits, re-solve requests	Blender 5.2 LTS extension (`blender_manifest.toml`); overlays drawn with the `gpu` module

The GPU stages run inside WSL2 by default. GEM-X documents Linux only, and ViPE and MHR build CUDA extensions that are fragile on native Windows. Blender, the proxy video and the project folder stay on Windows; the folder is shared through `\\wsl$` . If every primary model builds natively during P0, WSL2 can be dropped.

Blender add-on and worker

- **Start-up.** On import, the add-on starts one `afv serve` worker and reuses it. The environment's Python is located through an absolute path set in the add-on preferences, and the worker is spawned with no console window.
- **CPU-only fast path.** The `smooth`, `contacts` and `retarget` stages are NumPy/CPU-only and never import torch, so a smoothing change previews in seconds.
- **Non-blocking progress.** The add-on reads worker output on a background thread and applies results through `bpy.app.timers`, so the Blender UI never freezes.

Stage pipeline

The canonical motion between pose3d and export is the SOMA skeleton: local rotations, root trajectory and named contact channels in a Y-up, metre-scale world. It is converted to the export rig only at the end.

#	Stage	Output	Notes
0	ingest	Media info, timestamp table, proxy, shot list, letterbox and overlay masks	Rotation metadata applied
1	detect_track	Person boxes with persistent IDs	You pick the two fighters; referee and crowd ignored; SAM 2 masks hold IDs through clinches
2	kp2d	2D keypoints per fighter	GEM-X's matched ViTPose77 observation model feeds pose3d. RTMW-l adds hand and face points and drives reprojection QC only. Swapping the GEM-X observation model voids its published accuracy
3	camera	Per-frame camera pose and focal length, gravity, ground plane	ViPE's DPVO + MoGe-2 path only; fighters and graphics masked out; `--static-camera` for locked-off shots

4	pose3d	SOMA motion per fighter in one shared world	GEM-X at 30 Hz in windows of ⩽ 120 frames; resampled back to source timestamps
5	contacts	Named contact channels per frame; root and leg fix for skate and penetration	Starts from GEM-X's own contact output, so IK is not applied twice; painted contacts override
6	smooth	Filtered motion, contacts held, smoothing eased off at strike peaks; retime applied	CPU-only; pinned poses applied here
7	hands_face (optional)	Fist preset or finger rotations	Fingers depend on EgoHand-v1's licence
8	export	Canonical humanoid rig; FBX / glTF / BVH	Engine presets; your characters are retargeted in the engine

Caching and re-runs

- **Cache key.** Each stage result is keyed by a hash of the stage, model version and weight hash, settings, upstream keys and frame range. If none of these change, the stage is skipped.
- **Edit layers.** Fighter choice, ID-swap fixes, painted contacts and pinned poses are saved as small JSON edits. They are inputs to the stage they affect, and they survive re-runs of earlier stages. An edit that no longer matches its frames is flagged, not silently dropped.
- **Range re-runs.** A range re-run is stored as a patch over its base result; the effective result is the base plus its patches in order.
  - contacts, smooth and hands_face re-run exactly on the range, with blended edges.
  - camera and pose3d widen the range to whole model windows. The new segment is then rigidly aligned (yaw and translation) to its neighbours before blending, so the world trajectory after the range doesn't jump.
  - Changing the fighter choice invalidates kp2d onward, because keypoints and camera masks depend on the tracks.
- **Chunks.** The camera solve runs on chunks of up to 60 s, stitched by rigid alignment over the overlap. pose3d uses GEM-X's own window length.

- **Safe writes.** Results go to a temp folder and are renamed into place.

Plugin interface

```
class Stage(Protocol):
    name: str                    # "pose3d"
    impl: str                    # "gemx@<commit>"
    env: str                     # "gemx": the pinned environment it runs in
    licences: LicenceRecord      # code, weights, training-data terms (NFR-12)
    Params: type[pydantic.BaseModel]
    def run(self, ctx: Ctx, inputs: dict[str, Result], params: Params) ->
Result: ...
```

Stages exchange typed results (`Tracks` , `Keypoints2D` , `CameraTrack` , `BodyMotion` , `Contacts`). Adapters convert each model's native output, so changing a model touches only its adapter.

GPU, determinism and failures

- **GPU.** Stages run one at a time on one GPU. Each model process exits before the next loads. On out-of-memory errors the batch halves and retries, then falls back to a reduced resolution.
- **Determinism.** Golden runs fix seeds, enable `torch.use_deterministic_algorithms` and turn TF32 off. Tolerances are set from two repeat runs in P0.
- **Quality checks.** Each stage writes per-frame QC: keypoint confidence, reprojection error, foot speed in contact, acceleration, strike peak speed. The add-on draws these as the confidence strip.
- **Failures.** Bad input fails fast with a readable message, and a model crash keeps all earlier results.

Technology choices

The stack is built from models released in 2025–26 whose code and weights allow commercial use of outputs, so animations are safe to ship in a game you might sell. NVIDIA's GEM-X is the core: it produces world-grounded motion on the SOMA skeleton, with its own matched 2D keypoint model and contact clean-up. Research models built on the non-commercial SMPL body model (GVHMR, WHAM, TRAM, CameraHMR) are not used at all.

Model per stage

Stage	Primary	Fallback	Avoid	Licence status
Person detection and tracking	RF-DETR S/M + IoU/Kalman tracker + SAM 2 masklets	GEM-X's YOLOX HumanArt detector	Ultralytics YOLO (AGPL copyleft)	RF-DETR N–L Apache-2.0; SAM 2 Apache-2.0 (confirm)
2D keypoints for pose3d	GEM-X's ViTPose77 (DINOv3-huge) observation model	— (swapping voids GEM-X's published accuracy)	Sapiens (CC BY-NC)	Weight terms unstated; confirm in P0
2D hands, face and QC	RTMW-l via rtmlib/ONNX	RTMPose-m	—	Code Apache-2.0; weights trained on 14 mixed-licence datasets: record and review
Camera motion	ViPE using only its DPVO + MoGe-2 path	DPVO + MoGe-2 directly	ViPE's MetricTrail (NVIDIA non-commercial), Unik3D (CC BY-NC-SA), MASt3R	NVIDIA code Apache-2.0; DPVO MIT; MoGe-2 to confirm
World-grounded body	GEM-X Body Motion	Contingency only: SAM 3D Body per frame + camera + own trajectory optimiser (about 4–6 weeks to build)	GVHMR, WHAM, TRAM	Code Apache-2.0 (some files OpenMDW-1.1); weights under the NVIDIA Open Model License
Contacts	GEM-X Contact/IK output + own floor and skate optimiser	Velocity and height heuristics + IK	—	Own code

Smoothing	Own sliding-window optimiser (velocity and acceleration terms, eased at strike peaks), CPU-only	Quaternion-safe One-Euro filter	SmoothNet	Own code
Hands	Fist pose preset; fingers from GEM-X Hand Pose	SAM 3D Body hand head	HaMeR, WiLoR	GEM-X hand weights (EgoHand-v1) are gated: confirm terms

Accuracy evidence

Published figures; lower is better. GEM-X's numbers were measured with its original ViTPose keypoints and its Contact/IK post-processing switched on.

Method	Benchmark	Result	Source
GEM-X Body Motion	EMDB-2: W-MPJPE / RTE / jitter / foot slide	267 mm / 2.45% / 34.2 / 4.2	GEM-X README
GVHMR (reference, not used)	EMDB-2: W-MPJPE / RTE / jitter / foot slide	277 mm / 2.0% / 16.7 / 3.5	GVHMR paper
SAM 3D Body, DINOv3	EMDB / 3DPW PA-MPJPE	38.2 mm / 33.8 mm	SAM 3D Body paper
CameraHMR (reference, not used)	EMDB / 3DPW PA-MPJPE	43.3 mm / 35.1 mm	SAM 3D Body paper

These benchmarks are mostly everyday motion filmed with handheld cameras. None of them measures fast striking or broadcast zoom, and that gap is why the Phase 0 spike runs on real fight shots. GEM-X is as accurate as GVHMR globally but has about twice the jitter, so the smoothing stage is required. Its documented limits shape the pipeline: input is normalised to 30 fps, windows are at most 120 frames, and clips are at most 60 s.

Skeleton and export rig

- **Canonical skeleton.** Internally the tool uses NVIDIA's SOMA (Apache-2.0): 77 body joints plus 25 per hand. This is GEM-X's native output, and it maps to and from MHR, SMPL-X and Anny. Verify the exact joint list in P0.
- **One export rig.** The tool exports a single canonical humanoid rig rather than retargeting to every character. It uses Mixamo-compatible bone names (prefix configurable), a T-pose rest pose, and a `root` bone where the engine needs one.
- **Retargeting happens in the engine.** Each engine retargets that rig onto your characters with its own tools: Unity's Humanoid avatar, Unreal's IK Retargeter, Godot's BoneMap with SkeletonProfileHumanoid.
- **SOMA to export rig.** The conversion is fixed. Rotations are transferred relative to the rest pose, extra spine joints are spread proportionally, and a leg IK pass keeps planted feet planted. NVIDIA's SOMA retargeter is worth evaluating in P2 before writing this.

Software and hardware

Layer	Choice
ML environments	WSL2 Ubuntu, CUDA 12.8, one pinned environment per heavy model (`gemx` , `vipe` , `sam3db`); PyTorch 2.10 for GEM-X
Windows side	Blender 5.2 LTS (pinned build), rtmlib/ONNX Runtime, the `afv` CLI front end
Video	FFmpeg 7 via PyAV
Optimisation	NumPy/SciPy for the CPU-only stages; PyTorch inside model environments
Export	Headless Blender with explicit exporter settings per preset; own BVH writer
Reproducibility	`models.lock` : commit, weight hash, and dated copies of each licence text and model card

GPU tier	Expectation

RTX 3060 12 GB / 4060 Ti 16 GB	Likely workable for Standard mode. GEM-X's tested setup is an A100 80 GB, so consumer-GPU fit is measured in P0
RTX 4070–4090 class	Comfortable; batch queues finish overnight
8 GB cards	Reduced resolution only, if GEM-X's ViT-huge keypoint model fits at all

ViPE reports 3–5 fps on one GPU, so the camera solve dominates runtime on moving-camera shots. GEM-X is days old, and its behaviour on consumer GPUs is undocumented. The SAM 3D Body contingency is verified as a demo in P0, but it is built only if GEM-X fails the P0 gate.

What fight footage changes

Problem in broadcast and film footage	Handling
Two fighters plus a referee, constant overlap	Track everyone, keep the two chosen IDs, and use SAM 2 masklets to hold identity through clinches and crossings. GEM-X runs once per fighter against one shared camera solve, so both land in the same world. Human3R, a joint multi-person model, is a candidate once its licence is confirmed.
Pans and zooms	ViPE estimates intrinsics per frame. Fighters and on-screen graphics are masked out of the camera solve so they don't read as camera motion. Whether ViPE handles continuous zoom well is a Phase 0 test item.
Short shots and cuts	Each shot is solved separately; a take never crosses a cut. Strikes that land across a cut are not recoverable.
Small, blurred fighters in wide shots	Crops are upscaled before 2D keypoints and SAM 3D Body (High mode). Prefer mid shots and slow-motion replays when choosing shots.

Fast strikes	Smoothing eases off around detected velocity peaks, so punches and kicks keep their snap. Retiming a slow-motion replay adds temporal detail.
Undercranked kung fu films	A per-shot speed factor (e.g. 0.8×) restores believable timing. It is set by eye in Blender, because the true camera speed is rarely known.
Gloves and wraps	Fingers aren't visible, so a fist pose preset replaces finger solving for boxing and MMA.
Clinch and ground work	Out of scope for v1. Entangled bodies break current per-person models, and the penetration fix only handles light contact.

Data model, file formats and export

Each source video gets one project folder, with a subfolder per queued shot. That folder holds a JSON manifest, a compressed NumPy file per stage result, and small JSON edit files. Nothing else is needed: no database, no server and no custom binary format.

Project folder

```
fight_night.afv/
  project.json                  # source info, shot list, queue, settings
  media/proxy.mp4, pts.npy      # 720p proxy for Blender, true frame
timestamps
  shots/014/
    shot.json                   # range, speed factor, fighters, source
title/URL + timestamps
    results/<stage>/<key>/
        result.npz              # float32 arrays, e.g. rot[F,J,4], root[F,3],
contact[F,C], conf[F,J]
        meta.json               # model + weight hash, params, inputs, QC
summary, runtime
        patches/                # range re-runs layered over this result
    edits/ids.json, contacts.json, pins.json
    exports/<fighter>.fbx | .glb | .bvh
```

NPZ with float32 arrays is plenty for shots of up to 60 s: one fighter's 60 s, 30 fps SOMA take is under 3 MB. Every array is indexed by source frame and carries timestamps, so variable-frame-rate phone video stays in sync with the proxy in Blender.

Core records

Record	Key fields
Clip	source path and sha256, codec, size, rotation, nominal fps, VFR flag, timestamp table, work range
Actor	role (fighter A or B), chosen track ID, merged track segments, locked bone lengths, height (estimated or entered)
StageResult	stage, implementation and weight hash, params, input keys, frame range, files, QC summary
Take	skeleton (SOMA or target rig), fps, speed factor, local rotations [F,J,4] as xyzw quaternions (Blender uses wxyz; one helper converts), root translation [F,3], root-motion mode, contacts [F,C] with named channels, source provenance
ExportRig	fixed SOMA-to-export-rig mapping, bone-name prefix, rest pose (T-pose), root bone on or off
ExportPreset	format, engine, root-motion mode, constant export fps, and the full Blender exporter argument set (below)

Command line

```
afv shots fight_night.mp4                    # detect cuts, write the shot
list
afv queue fight_night.afv --shot 14 --shot 22 --range 31.2:36.8
afv run fight_night.afv --shot 14 --fighters auto --preset standard
afv run fight_night.afv --shot 22 --speed 0.25  # slow-motion replay back to
real time
afv run fight_night.afv --shot 14 --from smooth --frames 40:90 --set
smooth.strength=0.6
afv export fight_night.afv --shot 14 --preset unreal --fps 30
```

```
afv batch fight_night.afv                    # run every queued shot
overnight
afv provenance fight_night.afv               # sources and timestamps behind
each export
afv models --check                           # every checkpoint, its
licences, no unknowns
```

Export formats and engine presets

Format	Carries	Used for
FBX (via Blender)	Skeleton, joint animation, root motion, optional face curves	Unity and Unreal import; Mixamo-style rigs
glTF 2.0 / GLB	Skin, joint animation, morph-target weights	Godot, web viewers, any glTF pipeline
BVH	Hierarchy, rotations, root translation	Quick inspection and debugging

Engine preset	Rig	Root motion
Unity, Humanoid	Mixamo-style, no root bone	World motion stays on Hips; in-place or root motion is chosen in Unity's clip import settings (Root Transform Position/Rotation, Bake Into Pose)
Unity, Generic	Adds a `root` bone	Horizontal translation and yaw on `root`, set as the Root node
Unreal	Adds a `root` bone at the floor	Horizontal translation and yaw on `root`; enable root motion on the animation
Godot	Adds a `root` bone	On `root`; point `AnimationMixer.root_motion_track` at it
Blender	No root bone	World motion on Hips

Internally, everything is right-handed and Y-up, in metres, seconds and local quaternions. Export goes through Blender, so conversion happens in two steps: Y-up data is brought into Blender's Z-up scene, then Blender's exporter applies the preset's settings. Vertical

motion stays on the pelvis in every preset. Attacker and defender are exported as separate files that share one world origin, so paired animations line up when both start at the same place.

Each preset stores the full exporter argument set, because Blender's defaults break game exports:

- `axis_forward` and `axis_up`
- `apply_scale_options`, with Unreal's ×100 handled once
- `add_leaf_bones=False`
- `bake_anim_simplify_factor=0`, because the default of 1.0 strips subtle keys
- primary and secondary bone axes
- `use_armature_deform_only=True`
- `bake_anim_use_all_actions=False`

The round-trip test asserts that no extra armature root node appears.

Cleanup workflow in Blender

Cleanup happens in Blender through a sidebar panel called "AFV". Blender already provides playback, the viewport, the graph editor and FBX export, so the add-on only adds what Blender lacks: the video overlay, a confidence strip, contact painting and re-solve buttons. A 5 s exchange should take about 20–30 minutes from picking the shot to an in-engine animation.

Workflow for a 5 s exchange

Step	You do	The tool does	Time
1. Pick shots	Scrub the long video and queue shots where both fighters are fully in frame; set a speed factor for slow-motion or undercranked shots	Detects cuts and builds the shot list; a one-page guide lists what makes a good shot	1 min
2. Solve	`afv batch on the queue`	Runs all stages; prints a QC summary	~1–2 min

3. Review	In Blender: AFV → Import take	Loads both fighters on SOMA rigs, sets the proxy video as camera background, matches the solved camera, draws the confidence strip	1 min
4. Fix	Work through red spans (next section)	Re-solves only the affected window	3–8 min
5. Retarget	Optional: preview on your own character with Blender's retarget tools	Converts SOMA to the canonical export rig; final retargeting happens in your engine	1 min
6. Export	AFV → Export, pick the engine preset	Writes FBX or GLB to your game project's import folder	10 s

What the add-on adds to Blender

Tool	Interaction	Under the hood
Video overlay	Camera view shows the source video behind the rig	Proxy as camera background; solved focal length and camera path
Confidence strip	Coloured markers on the timeline for low keypoint confidence, foot slide or high reprojection error; Next issue jumps between them	Reads per-frame QC from the stage results
Pick fighters	Click the two fighters' numbered boxes; swap IDs where tracks crossed	Writes the fighter choice and ID fixes as an edit layer; re-runs from kp2d
Contact painting	Select a foot, scrub, press Plant or Free to mark a span	Writes `edits/contacts.json`; re-runs contacts and smooth on that window
Pin pose	Pose a bone with Blender's normal tools, then Pin	Stores the pose in `edits/pins.json`; smooth re-optimises ±N frames to blend into it
Smoothing	Global and per-region sliders, then Apply	Re-runs smooth only, about 5 s for 10 s of motion
Re-run range	Set a frame range, choose a stage and a preset (for example High)	`afv run --from <stage> --frames a:b`, blended at the edges

Hands off	Mark a span where hand data is bad	Blends to a relaxed hand pose

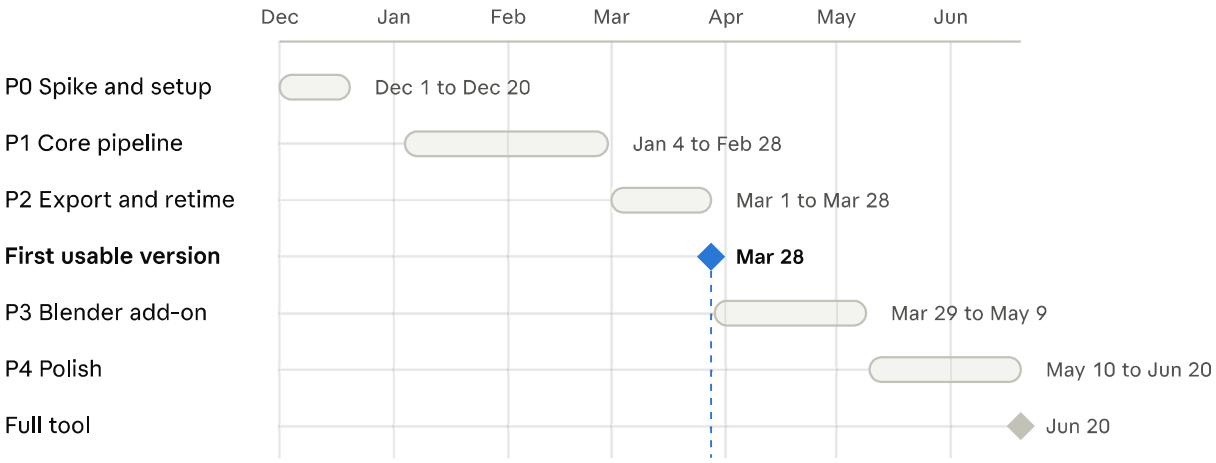
Final touches use Blender's own timeline, graph editor and NLA, and the add-on does not duplicate them. Solved motion lives on a base NLA action that every re-solve replaces. Your manual keyframe tweaks go on a separate additive layer above it, so they survive re-solves. Use pins instead when the solver itself should respect a fix.

Development plan

One developer working about 10–12 hours a week can have a first usable version by 2027-03-28 and the full tool by 2027-06-20. That is about 27 working weeks, or 300–400 hours of real effort, which needs a steady 12 hours a week. At 8 hours a week, expect the full tool around September 2027. Re-baseline the dates at every gate from the hours you actually logged. Each phase ends with something you can use and a gate you can check yourself. The riskiest question is answered first, in three weeks: do the chosen models hold up on broadcast and film fight footage, with two fighters, zoom and blur?

A usable pipeline by late March, the full tool by late June 2027

One developer at about 12 hours a week; each phase closes on its gate



The late-December gap is a holiday break. Everything after the first usable version makes cleanup faster and output better, but you can already make game animations with the CLI and Blender at that point.

Phases and gates

Phase	Dates	Build	Gate (all must pass)
P0 Spike and setup	2026-12-01 to 2026-12-20	Day 1: record GPU and VRAM, and fall back to WSL2 for the ML environment if a repo won't build natively within 4 h. Then `models.lock` with dated licence copies; run RF-DETR + SAM 2, RTMW, ViPE and the GEM-X demo on fight shots; assemble and hand-label the 12-shot test set; run 3 shots through one cloud mocap tool as a baseline	All primary models run; peak VRAM and runtime recorded per model; both fighters tracked without swaps on ⩾ 8 of 12 shots; ViPE separates zoom from motion on shot 3; ⩾ 4 of 6 shots rated "usable after cleanup". If GEM-X fails, stop and re-plan
P1 Core pipeline	2027-01-04 to 2027-02-28	Shot list and queue; detect/track, kp2d, camera and GEM-X wrapped as stages; two fighters in one world; contact fix; strike-aware smoother; whole-stage cache; QC numbers; batch mode	Test set solves unattended overnight; contact recall ⩾ 90% and slide ⩽ 2 cm on labelled plants; strike sharpness ⩾ 80%; ⩽ 1 ID swap per shot on shots 7–8
P2 Export and retime	2027-03-01 to 2027-03-28	One canonical humanoid export rig (Mixamo-compatible names, T-pose rest, plus a `root` bone where the engine needs it); engine presets; retime; fist preset; overlay and letterbox masks; FBX/GLB/BVH via headless Blender; paired attacker/defender export with a shared origin; round-trip test	Three exchanges play as paired animations on your characters in your engine, retargeted there; in-engine end displacement within 5% of the solve; round-trip test green. **First usable version**

P3 Blender add-on	2027-03-29 to 2027-05-09	Shot browser, import takes, video overlay, confidence strip, ID-swap fix, contact painting, pins, smoothing panel; edit layers, frame-range re-runs and chunking (needed by the add-on)	Median ⩽ 30 min from picking a shot to in-engine, stopwatch-timed over 3 new exchanges, using only the add-on and Blender
P4 Polish	2027-05-10 to 2027-06-20	Inter-person penetration fix; fingers for open-hand forms (if EgoHand-v1's licence allows); loop and blend helper for idles and footwork	Gloves don't pass through the opponent on shots 7 and 10; fingers usable on shot 11; footwork idle loops cycle with no visible pop

The last two weeks of December are left as a holiday gap. Each later phase carries about one week of slack inside its dates.

Backlog after P4

Work these in whatever order your game needs them:

- High mode: fuse SAM 3D Body for small or blurred fighters. This is research, with no published numbers yet.
- Clinch and grappling
- Linking one fighter across shots for longer sequences
- Physics plausibility pass
- Live preview into the engine

Ways to cut scope if time runs short

1. Skip P3's pinned poses and use Blender keyframes on the baked take instead.
2. Drop P4 entirely. Standard mode, the add-on and Blender's own tools already cover striking animation.
3. Defer fingers. Fist presets cover boxing and MMA, and only open-hand kung fu forms need finger solving.

Cost

The software is free. The main cost is your GPU. If your card has less than 12 GB of VRAM, the options are to run Standard mode only at reduced resolution, or to upgrade to a 16 GB-plus card.

Testing and quality checks

Without ground-truth mocap, quality is judged three ways:

- a fixed set of 12 fight shots
- a few hand-labelled facts about those shots
- your own eye in Blender and in-engine

The hand labels make the gates honest. A foot-skate number measured only on contacts that the tool detected itself can be driven to zero by the foot lock, whatever the real quality.

Test set (assembled in P0)

#	Shot	What it stresses
1–2	Boxing, wide shot, footwork and jabs	Foot contacts, two fighters, small figures
3	Boxing, mid shot with a broadcast zoom	Zoom versus motion
4	Boxing slow-motion replay of a hook	Retime, fast arm
5–6	MMA, kicks at range, cage camera	High legs, fast pans
7	MMA exchange into a clinch	ID holding, close contact
8	MMA with the referee crossing the fighters	Third person, occlusion
9–10	Kung fu film, staged exchange in a wide take	Choreography, undercranking
11	Kung fu film, open-hand form, one fighter	Fingers, flowing motion
12	Kung fu film, cut on impact	Shot boundary handling

Hand labels (about 2 hours, once)

- On shots 1, 2, 5 and 9, mark foot plant spans by frame.
- On every shot, mark the frames of 3 strike peaks.
- On shots 7 and 8, mark the true fighter IDs every 10 frames.

Metrics

Metric	How it is measured	Target
Contact recall	Labelled plant spans that the tool also detects	⩾ 90%
Slide within a plant	Maximum horizontal foot travel inside each labelled plant span	⩽ 2 cm
Strike sharpness	Wrist or ankle speed at labelled peaks after smoothing, as a share of the speed before	⩾ 80%
ID swaps	Swaps against the labelled IDs on shots 7 and 8	0 after the add-on fix; ⩽ 1 per shot before
Ground penetration	Mean foot depth below the floor during plants	⩽ 1.5 cm
Reprojection error	Solved joints projected onto the video against 2D keypoints	⩽ 15 px mean at 1080p
Root fidelity in engine	End-of-clip displacement in-engine against the solved trajectory	Within 5%
Speed	Wall-clock time per stage and in total, with peak VRAM	Recorded in P0; NFR-1 is then fixed on your GPU

Automated tests

- **Unit tests** cover rotation maths, change of basis, the export rig mapping, filters, retiming and the BVH writer.
- **Golden-file regression** runs every stage on 3 short shots and compares the results with stored outputs. Tolerances are 2D ⩽ 1.5 px and 3D ⩽ 2 mm mean. A model update is accepted by accepting new goldens.

- **Export round-trip.** Headless Blender re-imports each FBX and GLB and samples joint positions on every frame. The test passes at ⩽ 1 mm, with exact frame count and fps, and no axis flips. Foot skate and penetration are measured again on the exported rig after the round-trip, because proportion changes are where skate comes back. Once per phase, check the result in your actual engine, either by hand or scripted (Unity batchmode, Unreal commandlet, Godot --headless).
- **Licence check.** `afv models --check` fails if any checkpoint, including transitive downloads, has an unknown or non-commercial licence.

Visual review and blind A/B

At each phase gate, view every test shot beside its video in Blender, then play it in-engine on your character. Rate each shot "usable", "usable after cleanup" or "redo" in `tests/review.md`.

For A/B comparisons, such as two smoothing settings or a new model, a script shuffles and renames the outputs. Record your ratings before revealing which output is which.

As a yardstick, P0 also runs 3 test shots through one free-tier cloud video-mocap tool and records its quality and cleanup time. That baseline answers whether 300 hours of building is worth it. Check that tool's terms before uploading copyrighted footage.

Risks and licensing

The largest risks are legal, not technical, and they only bite if a game is sold. The first is where the footage comes from and what it captures: broadcast fights and films are copyrighted, and film fight choreography may be protected. The second is which models produced the animation. The largest technical risk is that the new models fall short on blurred, overlapping fighters, and the Phase 0 spike tests exactly that. Nothing here is legal advice. Before selling a game that uses extracted animation, have a lawyer look at the specific sources.

Source footage and the animations you extract

Question	Current understanding	What to do

Downloading from YouTube	YouTube's Terms of Service prohibit downloading except through YouTube's own features or an explicit download link. Third-party downloaders break the terms, which is a contract issue for your account rather than the copyright question below.	The tool takes local files and does not download. How you obtain footage is your call; know that this is the rule.
Copyright in the video	The broadcaster, league or studio owns the footage. Processing it locally for reference is low risk; redistributing the video is not part of this tool.	Keep source videos out of game repos and builds.
Motion from sports	Athletic movements in competition are generally treated as not copyrightable choreography. Generic jabs, hooks and kicks are very low risk.	Prefer sports footage for core move sets.
Motion from kung fu films	Fight choreography can be protected as a choreographic work. Whether extracted motion data infringes is unsettled; in Hanagami v. Epic Games (9th Cir. 2023), a claim over a short choreographed sequence used as a game emote was allowed to proceed.	For a sold game, don't reproduce a recognisable choreographed sequence as-is. Use extracted motion as reference: edit it substantially, split and recombine moves, or re-perform key moves yourself.
Likeness of famous fighters	Anonymous motion on your own character is rarely identifiable. Pairing a famous fighter's signature move with their name or look raises right-of-publicity issues.	Don't name or model the athlete; keep moves generic.
Provenance	If a source becomes a problem later, you need to know which takes came from it.	Every take records its source file, title or URL, and timestamps in its metadata, and `afv provenance` lists them per export.

Model licences in plain terms

Component	Licence	What it means for your games
SMPL, SMPL-X, MANO, FLAME body models	Non-commercial: prohibits producing artifacts for commercial purposes	Never on the path to a game asset. GVHMR, WHAM, TRAM, CameraHMR and HaMeR are dropped from the toolchain
GEM-X + SOMA	Code and SOMA Apache-2.0; weights under the NVIDIA Open Model License	Commercial use allowed; NVIDIA claims no ownership of outputs; the licence is revocable and ends if its guardrails are circumvented
SAM 3D Body + MHR	SAM License; MHR Apache-2.0	Royalty-free use with no non-commercial limit; restrictions on military and trade-controlled uses; outputs provided as-is
Transitive checkpoints: YOLOX HumanArt detector and EgoHand-v1 (GEM-X), MoGe-2 (ViPE)	Not yet established	Check in P0; finger solving depends on EgoHand-v1's licence
RTMW, RF-DETR (N–L), ViPE, DPVO, SAM 2	Apache-2.0 / MIT	Fine; skip ViPE's Unik3D path (NC-SA)
BEDLAM, AMASS (training data)	Non-commercial	Never fine-tune on them

Keep a dated copy of each licence text and model card next to its weight hash in `models.lock`. `afv models --check` must list every downloaded checkpoint, including transitive ones, with no licence marked "unknown".

Risk register

#	Risk	Likelihood	Impact	Mitigation

R1	A sold game contains motion that infringes film choreography or a fighter's publicity rights	Medium	High	Prefer sports footage; edit and recombine film moves; keep provenance per take; legal check before release
R2	Tracking swaps or merges the fighters in clinches and crossings	High	High	SAM 2 masklets; ID-swap fix in Blender; choose shots with separation; clinch work deferred
R3	GEM-X or SAM 3D Body fall short on small, blurred broadcast fighters	Medium	High	P0 spike on real fight shots before building; if GEM-X fails the P0 gate, stop and re-plan, because the fallback path adds about 2 months
R4	Model repos don't build on native Windows (ViPE, DPVO and LieTorch CUDA extensions)	High	Medium	Day 1 of P0: if a repo fails to build natively within 4 h, move the ML environment to WSL2 Ubuntu; Blender stays on Windows and calls `wsl afv …`
R5	Your GPU lacks the VRAM	Medium	Medium	Record GPU model and VRAM on day 1 (`nvidia-smi`); reduced-resolution mode for 8 GB; upgrade only if needed
R6	Zoom and fast pans corrupt the camera solve and world trajectory	Medium	Medium	Mask fighters and graphics from the camera solve; static-camera flag for locked-off shots; pick shots with gentle camera moves
R7	Smoothing dulls strikes, or impacts look floaty	High	Medium	Strike-aware smoothing; a peak-speed check in QC; hand-tune impacts in Blender
R8	Undercranked film gives wrong timing	High	Low	Per-shot speed factor set by eye

R9	Effort overruns, as is normal for a solo part-time project	High	High	Re-baseline dates at every gate from logged hours; apply the scope-cut list early
R10	Rapid model churn	High	Low	Swap a stage's adapter, then re-run the goldens and the visual review

Design review findings and resolutions

Two independent reviewers examined the draft: a technical reviewer (computer vision and game pipelines) and a planning and licensing reviewer. They raised 30 findings, 10 of them High, which are merged into the 24 rows below. All are resolved or tracked. The largest changes were letting each engine do the retargeting, defaulting the model environments to WSL2, and correcting several licence and root-motion claims. Separately, two scope changes from you reshaped the doc: the tool is personal, not a product, and its input is broadcast and film fight footage.

How the spec was produced

1. Four design agents drafted in parallel: ML research, architecture, product and UX, and delivery and QA.
2. The draft was then re-scoped twice: first from a commercial product to a personal tool for your own games, then from self-filmed clips to broadcast and film fight footage.
3. Two review agents checked it against primary sources: model cards, licence files, papers and engine docs.

Findings and resolutions

#	Severity	Finding	Resolution
1	High	Effort was underestimated by about 1.5–2× for one part-time developer	Re-estimated at 300–400 h at 12 h/week, with a slip estimate for 8 h/week; dates re-baselined at every gate; scope-cut list

2	High	Retargeting to every character duplicates what Unity, Unreal and Godot already do	One canonical humanoid export rig; retargeting happens in the engine
3	High	GEM-X expects its own ViTPose77 keypoints; RTMW is not a drop-in	GEM-X's matched observation model feeds pose3d; RTMW is used for hands, face and QC only
4	High	Weight licences were judged by repo licence; ViPE pulls in non-commercial weights; RTMW was trained on mixed-licence data	ViPE restricted to its DPVO + MoGe-2 path; NFR-12 records code, weight and training-data terms per checkpoint
5	High	Windows support for GEM-X, ViPE and MHR is undocumented; one shared environment won't hold	WSL2 by default, with one pinned environment per heavy model called as a subprocess
6	High	Frame-range re-runs were ill-defined for whole-sequence models	Patch-over-base results; camera and pose3d widen to whole windows and rigidly align to neighbours
7	High	Root-motion presets were wrong for Unity Humanoid and Mixamo	Per-engine table rewritten: Humanoid keeps motion on Hips; Unreal, Godot and Unity Generic get a `root` bone
8	High	P1 was over-built	Edit layers, range re-runs and chunking moved to P3, where the add-on needs them
9	High	The fallback pipeline was unbudgeted	Now a contingency: demo verified in P0, built only if GEM-X fails the gate
10	High	The first usable version lacked what game work needs (originally locomotion loops)	Superseded by the fight-footage scope; the P2 gate is now paired exchanges playing in-engine
11	Medium	The comparison preset with SMPL models was a licence grey area	Dropped; compare against published numbers only

12	Medium	Licence wording for the NVIDIA Open Model License and SAM License was imprecise; transitive checkpoints were missing	Wording sharpened; YOLOX HumanArt, EgoHand-v1 and MoGe-2 added; dated licence copies kept in `models.lock`
13	Medium	The Blender subprocess design couldn't meet the 5 s preview target	Long-lived `afv serve` worker; CPU-only smooth and contacts; non-blocking UI
14	Medium	Blender's exporter defaults break game exports; a "single change of basis" is impossible through Blender	Full exporter argument set stored per preset; round-trip test asserts no extra root
15	Medium	Blender 4.2 LTS is out of support	Pinned Blender 5.2 LTS build, packaged as an extension
16	Medium	Keyframe tweaks would be overwritten by re-solves	Base NLA action plus a separate additive tweak layer
17	Medium	Throughput target ignored ViPE's 3–5 fps camera solve	NFR-1 split into static and moving camera; measured on your GPU in P0
18	Medium	GEM-X operating limits (30 fps, 120-frame windows, 60 s clips) weren't reflected	pose3d runs at 30 Hz in GEM-X windows; camera chunks ⩽ 60 s
19	Medium	Gates were subjective or self-referential (foot skate measured on self-detected contacts)	Hand-labelled plants, strike peaks and IDs; stopwatch-timed gates; scripted blind A/B
20	Medium	High mode is research, not a feature	Moved to the backlog
21	Medium	Accuracy rows lacked sources; skeleton description inconsistent	Source column added; SOMA joint list to be verified in P0
22	Medium	Section inconsistencies: requirement counts, scope versus phases, time targets	Fixed during the rewrite
23	Low	Skate returns after retargeting; manual engine checks only; goldens need determinism	Metrics re-measured after the round-trip; optional scripted engine import; deterministic golden mode

24	Low	Existing tools could save time	P0 baseline run on a free-tier cloud tool; NVIDIA's SOMA retargeter evaluated in P2

Open items for P0

- [ ] Record the GPU model and VRAM, and decide native Windows or WSL2
- [ ] Confirm licence terms for GEM-X's ViTPose77, YOLOX HumanArt and EgoHand-v1 weights, plus MoGe-2 and SAM 2
- [ ] Test whether ViPE separates broadcast zoom from motion
- [ ] Measure GEM-X on fight shots: tracking, blur, small fighters
- [ ] Verify the SOMA joint list and the GEM-X contact channels